Part I, Week 01, Day 06 — Jacobian & Hessian

আজকের মূল আইডিয়া **(Day 05** এর সাথে কানেকশন**)**

- **Day 05:** Scalar function (একটা সংখ্যা আউটপুট) হলে `grad f = ∇f` বলে দেয় কোন দিকে গেলে সবচেয়ে দ্রুত `f` বাড়ে (steepest ascent)।
- কিন্তু Deep Learning-এ অনেক সময় একটা লেয়ার আউটপুট দেয় একটা ভেক্টর (অনেকগুলো activation একসাথে)।
- তখন "ডেরিভেটিভ" আর একটিমাত্র সংখ্যা/ভেক্টর থাকে না—এটা হয়ে যায় ম্যাট্রিক্স।

আজ আমরা শিখছি:

1. **Jacobian matrix**: ভেক্টর-ভ্যালুড ফাংশনের first-order derivative
2. **Hessian matrix**: second-order derivative (curvature বোঝার জন্য)

1) Jacobian Matrix: Vector-valued derivatives

1.1 Vector-valued function কী?

ধরি ইনপুট `x` হলো `n`-dimensional ভেক্টর:

```
x = [x1, x2, ..., xn]^T
```

এবং আউটপুট `y` হলো `m`-dimensional ভেক্টর:

```
y = f(x) = [y1, y2, ..., ym]^T
```

এখানে প্রতিটা `yi` আসলে `x1..xn` এর উপর depend করতে পারে।

1.2 Jacobian আসলে কী মাপে?

Jacobian বলে দেয়:

- আউটপুটের প্রতিটি component `yi`
- ইনপুটের প্রতিটি component `xj` পরিবর্তনে
- কতটা পরিবর্তিত হচ্ছে (sensitivity)

অর্থাৎ সব possible partial derivative একসাথে সাজানো:

```
J[i,j] = d(yi)/d(xj)
```

1.3 Jacobian matrix-এর শেপ (Shape)

- `y` এর dimension = `m`
- `x` এর dimension = `n`
- তাই Jacobian এর shape হবে: `m x n`

1.4 Jacobian-এর ম্যাট্রিক্স ফর্ম (Math-safe)

Jacobian `J`:

```
J =
[ dy1/dx1  dy1/dx2  ...  dy1/dxn
  dy2/dx1  dy2/dx2  ...  dy2/dxn
   ...      ...     ...    ...
  dym/dx1  dym/dx2  ...  dym/dxn ]
```

1.5 Backpropagation-এ Jacobian কেন দরকার?

Deep neural network এ আমরা বারবার **function composition** করি:

- Layer 1: `f`
- Layer 2: `g`
- পুরো নেটওয়ার্ক: `g(f(x))`

Scalar case-এ chain rule:
```
d/dx (g(f(x))) = g'(f(x)) * f'(x)
```

Vector case-এ ডেরিভেটিভগুলো **Jacobian**, তাই chain rule হয়:

```
J_{g o f}(x) = J_g(f(x)) * J_f(x)
```

খেয়াল করুন**:**

- এখানে `*` মানে **matrix multiplication**
- order খুব গুরুত্বপূর্ণ (কমিউটেটিভ না)

আপনার লাইনের “`Jg × Jf`” এর Math-safe লেখা:

```
J_total = J_g * J_f
```

1.6 Intuition (সহজ ব্যাখ্যা)

- `J_f` বলে: `x` একটু বদলালে `f(x)` (Layer-1 output) কেমন বদলায়
- `J_g` বলে: Layer-1 output একটু বদলালে final output কেমন বদলায়
- দুটো একসাথে গুণ করলে: `x` বদলালে final output কেমন বদলায়—এটাই backprop-এর core engine

আপনার জন্য “Plan” (Math + ML Safe Steps)

শেখার প্ল্যান

1. **Vector-valued function** সেট করুন: `y = f(x)`
2. **Dimension** লিখুন: `x in R^n`, `y in R^m`
3. **Jacobian define** করুন: `J[i,j] = d(yi)/d(xj)`
4. **Jacobian shape** মনে রাখুন: `J is m x n`
5. **Chain rule (matrix form)**:
   `J_{g o f}(x) = J_g(f(x)) * J_f(x)`
6. **Backprop mapping**:
   প্রতিটা layer-এর local Jacobian multiply হয়ে gradient propagate করে

Quick mini-example (optional, কিন্তু বুঝতে কাজে দেয়)

ধরি:
`x = [x1, x2]^T`

এবং
```
y1 = x1^2 + x2
y2 = 3*x1 - 5*x2
```

তাহলে Jacobian:

- `dy1/dx1 = 2*x1`, `dy1/dx2 = 1`
- `dy2/dx1 = 3`, `dy2/dx2 = -5`

So,

```
J =
[ 2*x1   1
  3    -5 ]
```

2) Hessian Matrix: Curvature

Gradient বনাম **Hessian — "ঢাল" বনাম "বাঁকানো"**

- **Gradient** `∇L(w)` বলে দেয়:
  "এই পয়েন্টে loss surface কোন দিকে সবচেয়ে দ্রুত বাড়ে/কমে?"
  (আপনি এটাকে **slope / velocity** ভাবতে পারেন)
- **Hessian** `H(w)` বলে দেয়:
  "এই ঢালটা কত দ্রুত বদলাচ্ছে?"
  অর্থাৎ surface টা কতটা বাঁকানো **(curvature / acceleration)**

একটা সহজ mental model:

- ঢাল (slope) = gradient
- ঢাল বদলানোর হার = Hessian (curvature)

2.0 Setup: Scalar loss + weights

ধরি loss function:

`L(w)` যেখানে `w` হলো `n`-dimensional weight vector:

```
w = [w1, w2, ..., wn]^T
```

এবং `L(w)` আউটপুট দেয় একটা **scalar** (একটা সংখ্যা), যেমন training loss।

2.1 Hessian কী?

Hessian হলো সব **second-order partial derivative** একসাথে সাজানো একটি matrix:

```
H[i,j] = d^2 L / (dwi dwj)
```

Hessian shape

যেহেতু `w`-তে `n`টা weight আছে, তাই:

`H` এর shape = `n x n`

2.2 Hessian matrix ফর্ম (Math-safe ASCII)

Hessian-এর ম্যাট্রিক্স ফর্ম

$$H = \begin{bmatrix} \frac{\partial^2 L}{\partial w_1^2} & \frac{\partial^2 L}{\partial w_1 \partial w_2} & \cdots & \frac{\partial^2 L}{\partial w_1 \partial w_n} \\ \frac{\partial^2 L}{\partial w_2 \partial w_1} & \frac{\partial^2 L}{\partial w_2^2} & \cdots & \frac{\partial^2 L}{\partial w_2 \partial w_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 L}{\partial w_n \partial w_1} & \frac{\partial^2 L}{\partial w_n \partial w_2} & \cdots & \frac{\partial^2 L}{\partial w_n^2} \end{bmatrix}$$

ইন্টুইশন:

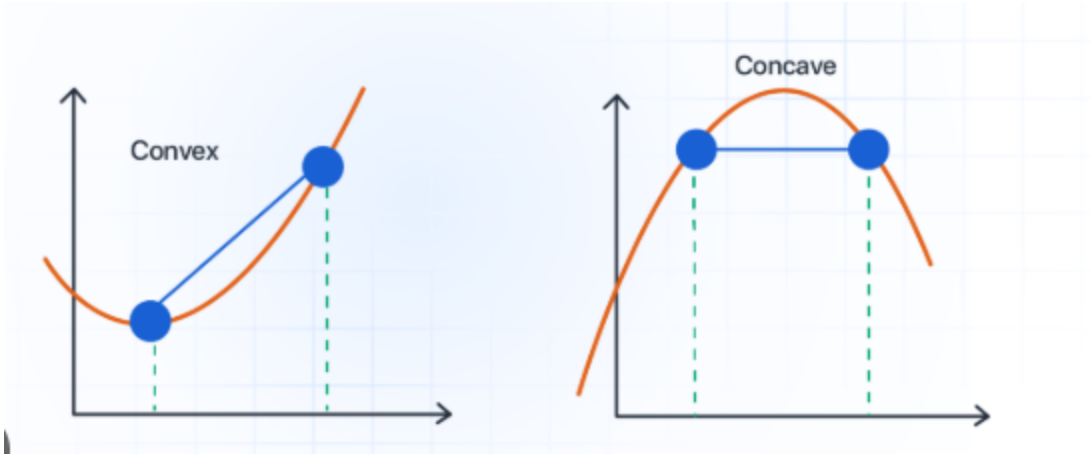
- diagonal entries $\frac{\partial^2 L}{\partial w_i^2} \rightarrow w_i$ direction-এ curvature কেমন
- off-diagonal entries $\frac{\partial^2 L}{\partial w_i \partial w_j} \rightarrow w_i$ এবং w_j এর interaction/coupling কতটা

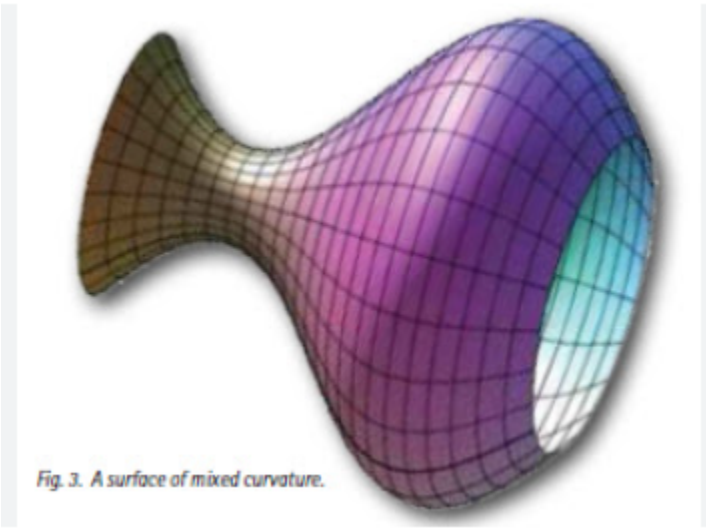
এখানে diagonal entries (`d^2L/dwi^2`) = প্রতিটা weight direction এ “local curvature”
off-diagonal entries (`d^2L/(dwi dwj)`) = weight `wi` আর `wj`-এর interaction / coupling

2.3 “Curvature” মানে আসলে কী? (ইন্টুইশন)

Loss surface যদি 2D হয় (শুধু `w1, w2`), তাহলে আপনি এটাকে পাহাড়/উপত্যকা কল্পনা করতে পারেন।

- **Positive curvature** (cup-shaped / convex up):
  নিচের দিকে একটা বাটি (bowl) টাইপ — minima থাকার সম্ভাবনা ভালো
- **Negative curvature** (cap-shaped / concave down):
  উল্টো বাটি — maxima/unstable region
- **Mixed curvature** (একদিকে up, অন্যদিকে down):
  saddle point (DL-এ খুব common)





Hessian এই curvature-কে সংখ্যায় মাপে।

2.4 Symmetry of the Hessian (Clairaut's Theorem)

$$\frac{\partial^2 L}{\partial w_i \partial w_j} = \frac{\partial^2 L}{\partial w_j \partial w_i}.$$

এটা সত্য হয় যদি `L` যথেষ্ট smooth হয় (continuous second derivatives আছে)।

ফলাফল:

- Hessian **symmetric**:

```
H = H^T
```

অর্থাৎ:
```
H[i,j] = H[j,i]
```

Symmetry কেন গুরুত্বপূর্ণ?

কারণ symmetric matrix-এর উপর Spectral Theorem apply হয়।

2.5 Spectral Theorem consequence (Day 03 কানেকশন)

যেহেতু Hessian symmetric, তাই নিশ্চিতভাবে:

1. Hessian-এর সব **eigenvalue real** (কাল্পনিক নয়)
2. Hessian-এর eigenvector গুলো **orthogonal** (একটা আরেকটার সাথে 90°)

Math-safe স্টেটমেন্ট:

- `H v_k = lambda_k v_k`
- `lambda_k` real
- `v_i^T v_j = 0` যখন `i != j`

2.6 Eigenvalues দিয়ে curvature বোঝা (সবচেয়ে কাজের অংশ)

Hessian-এর eigenvalues `lambda` আসলে curvature-এর “principal directions” এ পরিমাপ।

- যদি সব `lambda_k > 0` হয় ⇒ `H` positive definite ⇒ point টা strict local minimum (convex bowl)
- যদি সব `lambda_k < 0` হয় ⇒ negative definite ⇒ local maximum
- যদি কিছু positive, কিছু negative ⇒ **saddle point**
- যদি কিছু `lambda_k = 0` (বা খুব ছোট) ⇒ flat direction / plateau (ill-conditioned)

Deep learning এ সাধারণত:

- অনেক eigenvalue ছোট (flat directions)
- কিছু বড় positive/negative (sharp directions / saddle curvature)

2.7 Optimization-এ Hessian কেন লাগে?

(A) Newton's Method (2nd-order optimizer)

Newton update (Math-safe):

```
w_{new} = w - H^{-1} * grad(L)
```

যেখানে:

- `grad(L)` = ∇L(w)
- `H^{-1}` = curvature অনুযায়ী step কে rescale করে

ইন্টুইশন:

- যদি curvature বড় (sharp) ⇒ step ছোট হওয়া উচিত
- যদি curvature ছোট (flat) ⇒ step বড় হতে পারে

Gradient Descent একই learning rate সবদিকে দেয়, Newton curvature দেখে "smart step" দেয়।

কিন্তু DL-এ Newton কেন কম?

- Hessian compute/store expensive: `n x n` (n অনেক বড়)
- Inversion expensive
- তাই practical DL-এ: quasi-Newton (L-BFGS), Hessian-vector product, Adam/SGD ইত্যাদি বেশি ব্যবহার হয়

3) Curvature, Eigenvalues, and Saddle Points

কেন Hessian দরকার?

Gradient Descent (GD) শুধু "এই মুহূর্তের ঢাল" দেখে:

- `grad L(w)` (অর্থাৎ `∇L`) tells slope / steepest direction
- কিন্তু GD জানে না ঢালটা সামনে গিয়ে কীভাবে বদলাবে (curvature blind)

তাই যদি loss landscape হয় **narrow ravine / long valley** (একদিকে steep, অন্যদিকে flat):

- GD একদিকে বেশি নামতে গিয়ে অন্যদিকে overshoot করে
- ফলে **zigzag** করে, convergence slow/chaotic হয়

Hessian `H` local curvature map দেয়—কোন direction এ কতটা বাঁকানো, কোথায় sharp, কোথায় flat।

3.1 Critical point কী?

Critical point মানে যেখানে gradient zero:

```
∇L(w*) = 0
```

এখানে তিনটা possibility:

- local minimum
- local maximum
- saddle point

Gradient একা দেখে এগুলো আলাদা করা যায় না, কারণ তিন ক্ষেত্রেই `∇L = 0` হতে পারে।

3.2 Eigenvalues দিয়ে geometry classify করা

Hessian symmetric হলে (সাধারণ smooth loss এ হয়), আমরা eigen decomposition ধরতে পারি:

```
H v_k = lambda_k v_k
```

এখানে:

- `lambda_k` = eigenvalue (curvature magnitude/sign)
- `v_k` = eigenvector (principal direction)

Interpretation (সবচেয়ে গুরুত্বপূর্ণ):

- প্রতিটা eigenvector direction `v_k` বরাবর curvature ~ `lambda_k`
- sign বলে দেয় direction টা “upward bowl” নাকি “downward” নাকি mixed

3.3 তিনটা কেস (Math-safe rules)

Case A: Positive Definite (সব eigenvalue positive)

শর্ত:

`lambda_k > 0` for all k

মানে:

- সব direction এ surface "উপরে দিকে বাঁকানো" (bowl-shaped)
- তাই point টা **Local Minimum**

ইন্টুইশন: চারদিক থেকেই নিচের দিকে নেমে এসে এখানে থামে।

Case B: Negative Definite (সব eigenvalue negative)

শর্ত:

`lambda_k < 0` for all k

মানে:

- সব direction এ surface "নিচের দিকে বাঁকানো" (upside-down bowl)
- তাই point টা **Local Maximum**

Case C: Indefinite (মিশ্র sign eigenvalues)

শর্ত:

কিছু `lambda_k > 0` এবং কিছু `lambda_k < 0`

মানে:

- কিছু direction এ bowl-like (upward curvature)
- কিছু direction এ peak-like (downward curvature)
- তাই point টা **Saddle Point**

ইন্টুইশন: ঘোড়ার পিঠের মতো—একদিকে উঠা, অন্যদিকে নামা।

3.4 GD কেন Saddle/Valley তে ঝামেলা করে?

1) Narrow ravine-এ zigzag

ধরি একটি direction এ curvature বড় (sharp) আর অন্যদিকে ছোট (flat):

- sharp direction এ step নিলে overshoot হয়
- flat direction এ progress ধীর হয়
  ফলে GD "এদিক-ওদিক" bounce করে এগোয়।

2) Saddle-এর কাছে "flat zone" অনুভূতি

Saddle point এ gradient zero হতে পারে:
`∇L = 0`

কিন্তু এটি minimum না—কারণ অন্তত এক direction এ `lambda < 0` থাকে, অর্থাৎ নামার রাস্তা আছে।
সমস্যা:

- gradient signal খুব ছোট/নেই ⇒ GD থেমে যাওয়ার মতো আচরণ করে
- বা very slow escape হয় (numerical noise ছাড়া)

3.5 High-dimensional (LLM/CNN) এ কেন Saddle বেশি?

আপনার নোটের বক্তব্যটা এইভাবে বোঝা যায়:

- dimension বড় হলে (weights মিলিয়ন/বিলিয়ন)
- critical point এ সব eigenvalue একসাথে positive হওয়া (pure minimum) তুলনামূলকভাবে কম ঘটে
- অনেক direction থাকে যেখানে curvature sign mixed হয়
  ⇒ saddle-type critical point প্রচুর

আমি এখানে "প্রুফ" দিচ্ছি না (নোটও দেয় না), কিন্তু DL optimization literature-এ এই intuition খুব common: high-dim non-convex landscapes এ saddle/plateau বেশি দেখা যায়।

3.6 Adam / Momentum কেন সাহায্য করে? (আপনার টেক্সটের লিঙ্ক)

Momentum ধারণা (conceptual):

- GD update:
  ```
  w <- w - eta * grad(L)
  ```

- Momentum adds “velocity” state `v`:

```
v <- beta*v + grad(L)
w <- w - eta * v
```

এতে লাভ:

- ravine-এ oscillation কমে (zigzag smooth হয়)
- flat zone / saddle vicinity-তে accumulated velocity থাকায় ধাক্কা দিয়ে বের হয়ে যেতে পারে
- noisy gradients থাকলে momentum directional consistency ধরে রাখে

Adam মূলত momentum + per-parameter adaptive scaling করে (দুইটা moving average), তাই বাস্তবে saddle/plateau তে often better behavior দেখা যায়।